UNIQ ID: E0124092

RECYCLING ROBOT - MDP IMPLEMENTATION

```
import random


states = ["High", "Low"]
actions = ["Search", "Wait"]
rewards = {
    ("High", "Search"): 2,
    ("High", "Wait"): 0,
    ("Low", "Search"): 1,
    ("Low", "Wait"): 0
}
transitions = {
    ("High", "Search"): {
        "High": 0.8,
        "Low": 0.2
    },

    ("High", "Wait"): {
        "High": 1.0,
        "Low": 0.0
    },
    ("Low", "Search"): {
        "High": 0.0,
        "Low": 1.0
    },

    ("Low", "Wait"): {
        "High": 0.6,
        "Low": 0.4
    }
}

print("===== RECYCLING ROBOT MDP =====")

print("\nStates:")
for state in states:
    print("-", state)

print("\nActions:")
for action in actions:
    print("-", action)


print("\nRewards:")

for (state, action), reward in rewards.items():
```

```
    print(
        f"State = {state}, "
        f"Action = {action}, "
        f"Reward = {reward}"
    )


print("\nTransition Probabilities:")

for (state, action), next_states in transitions.items():

    print(f"\nState = {state}, Action = {action}")

    for next_state, probability in next_states.items():
        print(
            f"  -> {next_state}: "
            f"{probability}"
        )

policy = {
    "High": "Search",
    "Low": "Wait"
}

print("\nPolicy:")

for state, action in policy.items():
    print(f"{state} -> {action}")

def take_step(state, action):

    reward = rewards[(state, action)]

    next_state_probabilities = transitions[(state, action)]

    next_states = list(next_state_probabilities.keys())
    probabilities = list(next_state_probabilities.values())
    next_state = random.choices(
        next_states,
        weights=probabilities,
        k=1
    )[0]

    return next_state, reward

print("\n===== ROBOT TRAJECTORY =====")

current_state = "High"

total_reward = 0
steps = 10

trajectory = [current_state]

for step in range(steps):
```

```
    action = policy[current_state]

    next_state, reward = take_step(
        current_state,
        action
    )

    print(f"\nStep {step + 1}")
    print("Current State :", current_state)
    print("Action        :", action)
    print("Reward        :", reward)
    print("Next State    :", next_state)

    total_reward += reward

    current_state = next_state

    trajectory.append(current_state)

print("\n===== TRAJECTORY =====")

print(" -> ".join(trajectory))

print("\nTotal Reward:", total_reward)
```

```
===== RECYCLING ROBOT MDP =====

States:
- High
- Low

Actions:
- Search
- Wait

Rewards:
State = High, Action = Search, Reward = 2
State = High, Action = Wait, Reward = 0
State = Low, Action = Search, Reward = 1
State = Low, Action = Wait, Reward = 0

Transition Probabilities:

State = High, Action = Search
  -> High: 0.8
  -> Low: 0.2

State = High, Action = Wait
  -> High: 1.0
  -> Low: 0.0

State = Low, Action = Search
  -> High: 0.0
  -> Low: 1.0
```

```
State = Low, Action = Wait
  -> High: 0.6
  -> Low: 0.4

Policy:
High -> Search
Low -> Wait

===== ROBOT TRAJECTORY =====

Step 1
Current State : High
Action        : Search
Reward        : 2
Next State    : Low

Step 2
Current State : Low
Action        : Wait
Reward        : 0
Next State    : Low

Step 3
Current State : Low
Action        : Wait
Reward        : 0
Next State    : High

Step 4
Current State : High
Action        : Search
Reward        : 2
Next State    : Low

Step 5
Current State : Low
Action        : Wait
Reward        : 0
Next State    : Low

Step 6
Current State : Low
Action        : Wait
Reward        : 0
Next State    : High

Step 7
Current State : High
Action        : Search
Reward        : 2
Next State    : High

Step 8
Current State : High
Action        : Search
```

```
Reward        : 2
Next State    : High

Step 9
Current State : High
Action        : Search
Reward        : 2
Next State    : High

Step 10
Current State : High
Action        : Search
Reward        : 2
Next State    : High

===== TRAJECTORY =====
High -> Low -> Low -> High -> Low -> Low -> High -> High -> High -> High ->
High

Total Reward: 12
```